

A Behavioural Anonymity Set for Solana

Batched zero-knowledge actions, and the funding-provenance
channel measured against live mainnet data

Thiago Rocha Duarte
thiagord2016@gmail.com

July 2026

Abstract. A privacy pool on a public ledger advertises an anonymity set of k members and, almost universally, reports k as the anonymity it delivers. This work presents a protocol that gives such a set to protocol actions rather than to funds, and measures the one channel that no protocol of this kind can close. Members deposit a fixed denomination; a member later proves in zero knowledge that they own some note in the accumulator and directs the pool to execute an arbitrary instruction on an arbitrary program, which the pool performs and signs, batched with the actions of other members under one signature at one timestamp. A proof verifies on chain for about 101,000 compute units, half the default budget of a single instruction, and the binding constraint on the size of a crowd turns out to be the transaction format rather than the cryptography: ten members per legacy settlement of payments, twenty through an address lookup table, and six when each member delegates stake to a validator of their own choosing, because behaviour that diverges costs anonymity-set size. The channel that remains open is funding provenance, since an observer may partition the members of any deposit-based set by the origin of their capital. Measured against live mainnet data on an unrelated and live privacy pool, the loss factor is 0.0955, inside an unresolved bracket of 0.0350 to 0.1136 and a sampling interval of 0.0848 to 0.1790, so that knowing a member's funding class costs that pool roughly an order of magnitude of its nominal anonymity. Of eight collection runs, six produced no publishable figure and are reported in full, along with a defect found in the measuring instrument itself and a cross-population comparison that the tool refuses to report although its point estimates favour the argument made here.

Keywords: Blockchain privacy; Anonymity set; Zero-knowledge proofs; Anonymity metrics; Funding provenance; Solana.

1 Introduction

This section states the problem the work addresses, the research question and its three sub-questions, the contributions, and how the rest of the article is organised. The technical concepts it uses in passing are defined in Section 2.

On a public ledger, the record that identifies a participant is rarely the amount they moved. It is the pattern of what they did. A staker’s choice of validator is public, stable across epochs and far more distinctive than the size of the stake; a voter’s governance history is a signature written in the clear; a trader’s routine is visible to anyone willing to read the ledger backwards. Privacy tooling for these ledgers, however, has been built almost entirely around value. A fixed-denomination pool of the kind described by Pertsev, Semenov e Storm (2019) breaks the link between a deposit and a withdrawal, which protects funds and leaves behaviour untouched: the member who withdraws still has to act, and the action still carries their signature.

Making an action unattributable demands something a value mixer does not provide. Somebody must sign as the authority for that action, and a delegation of stake or a governance vote cannot be expressed as a transfer. For a member who must never appear on chain, the only candidate signer is the pool itself. This is the first half of the problem, and it is a matter of protocol design.

The second half cannot be designed away. Any deposit-based anonymity set on a public ledger can be partitioned by where each member’s capital came from. An observer who learns a member’s funding class no longer has to guess among all k members of the set, only among the members of that class, so the anonymity that survives is the size of the class rather than k . No pool controls where the money of its users came from, and no improvement to a circuit closes the channel. Empirical work on deployed systems has repeatedly found the delivered anonymity to be below the advertised figure, on Zcash (KAPPOS *et al.*, 2018), on Monero (MÖSER *et al.*, 2018) and for the users of a fixed-denomination pool on Ethereum (BERES *et al.*, 2021). What has not been done, as far as this work could establish, is a measurement of the funding-provenance channel against live Solana data with a published method.

Those two halves are the subject of this article, and they are treated as one problem. A protocol that closed the action side and reported a nominal k would be advertising an anonymity it has not measured. A measurement

without a protocol would describe a channel nobody had tried to act on. The claim made here is deliberately narrow and has two parts: the action side is closed, and the membership side is measured.

The research question follows:

Can a Solana protocol give an anonymity set to arbitrary protocol actions rather than to funds, with on-chain zero-knowledge verification and a shared settlement timestamp; what bounds the size of such a crowd in practice; and how much of the nominal anonymity k survives the funding-provenance channel when that channel is measured against live mainnet data rather than modelled?

Three sub-questions organise the evaluation. First, what does it cost, in compute units and in transaction bytes, to verify a membership proof on chain and to settle the actions of n members in one transaction, and which of the two limits binds first. Second, what loss factor a real Solana privacy pool exhibits under a passive, chain-only adversary, and how much of the uncertainty in that figure comes from members the tracer failed to resolve rather than from the pool itself. Third, whether the depositors of a privacy pool are measurably more concentrated in provenance than an ordinary population that is not seeking deniability at all. The third is included because the honest answer to it, on the evidence collected here, is no, and reporting that is part of the contribution rather than a failure of it.

The contributions are five.

1. **A protocol that gives an anonymity set to actions rather than to funds.** Members deposit a fixed denomination; a member later proves in zero knowledge that they own some note in the accumulator and directs the pool to perform an arbitrary instruction on an arbitrary program. The proof is verified by the chain, the action is executed and signed by the pool, and the whole batch settles in a single transaction under one signature and one timestamp (Section 3).
2. **Measured ceilings on the size of a crowd**, taken by serialising real instructions and by settling real batches on a live cluster, under two transaction formats and three shapes of action. The result is that the binding constraint is the transaction format rather than the cryptography, and that escaping one format limit is paid for out of the other (Section 5.3).
3. **An instrument for measuring the funding-provenance channel**, with a stated adversary, a metric chosen for comparability across pools of different sizes, a member-weighted sampling frame, a failure taxonomy that never lets an infrastructure limit read as evidence, and gates that

refuse to publish a headline from a run that does not support one (Section 4).

4. **Eight collection runs against live mainnet data**, of which six produced no publishable figure and are reported in full, together with a defect found in the instrument itself and the re-collection it forced, and a cross-population comparison that the tool refuses to report although its point estimates favour the argument of this work (Section 5.4 to Section 5.6).
5. **An explicit set of non-claims** (Section 6.3), including the two that matter most: the trusted setup used here is reproducible and not secure, and the funding-provenance channel is not closed.

The rest of the article is organised as follows. Section 2 introduces the execution model, the mixing literature, the anonymity metrics this work reports and the chain-analysis heuristics it relies on, and positions the work against each group. Section 3 presents the protocol. Section 4 presents the measurement method. Section 5 evaluates both, giving costs and crowd ceilings first and the provenance runs second. Section 6 discusses what the two halves buy together, states the limitations and threats to validity, and lists what the work does not claim. Section 7 concludes and names the work that follows from it.

2 Background and Related Work

This section gives the concepts the rest of the article uses, in the order it uses them: the execution model whose limits bound a crowd (Section 2.1), the mixing literature the protocol inherits from (Section 2.2), the anonymity metrics the measurement reports and one widely repeated formula that is inverted (Section 2.3), and the chain-analysis heuristics the tracer is built on (Section 2.4). Section 2.5 positions this work against each group. The notation fixed in Section 2.3 is used unchanged for the remainder of the article.

2.1 Public ledgers and the Solana execution model

A public permissionless ledger publishes every transaction, its signers, the accounts it touches and the balance of each account before and after. Nothing is forgotten and anyone may read it retrospectively, which is what makes behavioural patterns durable evidence rather than transient observations.

Solana (YAKOVENKO, 2018) organises state as accounts owned by programs. A program may call another program through a cross-program invocation (CPI), and may sign for accounts derived from its own identifier,

called program derived addresses (PDAs), without holding a private key. Four platform limits matter to this work, and all four are properties of the transaction rather than of any program. A transaction must fit in a 1232-byte packet (SOLANA FOUNDATION, 2026c). A legacy transaction names every account by its full 32 bytes; a versioned transaction may instead resolve most accounts through an address lookup table (ALT), an on-chain account listing addresses that the transaction then references by one byte each (SOLANA FOUNDATION, 2026a). Execution is metered in compute units (CU), with a default budget per instruction that a transaction may raise by including an explicit compute-budget instruction (SOLANA FOUNDATION, 2026b). Finally, a transaction takes a lock on every distinct account it names, and the number of such locks is capped.

These are not incidental. Section 5.3 shows that the number of members who can share one settlement is set by these limits and not by the cost of the cryptography, and that raising one of them is paid for out of another.

One further detail of the model is load-bearing for Section 2.4: the first account key of a transaction is both the fee payer and a required signer. That makes “who signed” a public fact about every transaction, which is a deanonymisation channel when the signer is a member and a clustering heuristic when it is an exchange.

Programs may also call a small set of cryptographic syscalls provided by the runtime, among them operations on the BN254 pairing-friendly curve (ANZA, 2026), the family introduced by Barreto e Naehrig (2006). This is what allows a proof to be verified by the chain itself rather than by a committee standing in for it.

2.2 Mixes, batching and deposit pools

The anonymity set is the oldest idea in this literature. Chaum (1981) introduced the mix, which collects messages from many senders and emits them in a form and an order that prevent an observer from matching output to input, so that each sender hides among the others. Everything that follows is a variation on how the collecting is done.

How a mix decides to flush its batch is where it is attacked. Serjantov, Dingledine e Syverson (2002) analyse threshold, timed, and threshold-or-timed batching, and show that the disjunction inherits the weakness of its weaker half: an adversary who can wait, or who can control when a batch is flushed, reduces the set to a single sender. Section 3.3 adopts exactly this disjunction, and Section 6.2 states the consequence rather than leaving it implicit.

Deposit-based pools brought the idea to public ledgers. The note model that Zerocash (BEN-SASSON *et al.*, 2014) established is the one used here: a

member deposits and records a commitment in an append-only accumulator; to spend, the member proves in zero knowledge that they own some leaf of that accumulator and reveals a nullifier, a value derived from the note’s secret that the program records so the same note cannot pay out twice. Neither the leaf nor the member is named. Pertsev, Semenov e Storm (2019) applied this with fixed denominations, so that deposits are mutually indistinguishable by amount.

Two failure modes of that design are worth naming because Section 3.1 is built to make them unrepresentable. If the value of a note is carried as a field of the note and is not bound into the commitment, a depositor of a small amount can withdraw a large one. If the nullifier is scoped to an epoch rather than being global, one deposit pays out once per epoch, indefinitely.

2.3 Anonymity metrics, and one formula that is quoted inverted

Two symbols for size are used throughout and are not interchangeable. **Nominal** k is the anonymity a pool advertises, namely the number of notes its accumulator holds. K is the number of members of the particular set being measured, which in Section 5.4 is the number of sampled depositors whose funding chain reached a class, and is therefore smaller.

Let K be the number of members of the measured set, X the random variable naming which member performed an observed action, and $C = \lambda(X)$ the class the observer assigns to a member under a labelling function λ . Let m be the number of observed classes and n_c the size of class c , so that $\sum_c n_c = K$. Because each member belongs to exactly one class, C is a deterministic function of X , $H(C | X) = 0$, and the chain rule collapses to

$$\log_2 K = H(C) + H(X | C) \tag{1}$$

The two terms divide the advertised anonymity into what the partition reveals and what survives it. Serjantov e Danezis (2003) define the anonymity of a distribution as its entropy. Their Definition 2 is explicit that the quantity is measured in bits and taken over users, and the paper does not state an exponentiated form; the exponentiated quantity, the scaled anonymity set size, is due to Andersson e Lundin (2008). Díaz *et al.* (2003) proposed a closely related normalised measure in the same year.

This distinction matters because a formula in wide circulation is inverted. The quantity $2^{H(C)}$, the exponentiated entropy of the class-size distribution, is often quoted as the effective anonymity set. It is not. It is maximised when

every member is alone in their own class, which is total deanonymisation, so it measures the leakage. The surviving anonymity is $2^{H(X|C)}$.

The headline figure of this work is therefore the loss factor

$$\rho = 2^{-H(C)} \quad (2)$$

the fraction of nominal k that survives the partition. It is reported instead of an effective- k because it does not depend on K and is therefore comparable between pools of different sizes, which an effective- k is not.

Two further metrics are reported beside it, because a single summary hides different things. Min-entropy leakage, in the sense of Smith (2009), measures the adversary’s single-guess success and is insensitive to the shape of the tail that Shannon entropy averages over. That averaging is not a theoretical concern: Tóth, Hornák e Vajda (2004) exhibit two systems with identical entropy of 4.3219 bits, one uniform over 20 users and one over 101 users in which the true sender has probability one half, where the adversary succeeds once in twenty in the first case and once in two in the second. Möser *et al.* (2018) established the template for reporting advertised against effective anonymity on a deployed system.

Finally, a measured class distribution is a sample, and two estimators say how much of it was seen. Good–Turing coverage (GOOD, 1953) estimates the fraction of the class mass observed from the number of classes seen exactly once, and Chao1 (CHAO, 1984) estimates the true number of classes from the counts of classes seen once and twice. Both are used in Section 5.4 as saturation diagnostics rather than as results. Interval estimates come from the percentile bootstrap (EFRON; TIBSHIRANI, 1993), and one bias is carried explicitly: the plug-in estimator of entropy is biased downward when classes are many and members few (PANINSKI, 2003), which for $\rho = 2^{-H(C)}$ means the reported figure is biased upward.

2.4 Chain analysis and funding provenance

The adversary of Section 4.1 needs to group addresses, and the heuristics that let them do so are well established. Meiklejohn *et al.* (2013) demonstrated on Bitcoin that addresses appearing as common inputs to one transaction are controlled by one party, and that a small amount of ground truth propagates through such clusters to name real services. The Solana analogues follow from Section 2.1: addresses that share a fee payer, addresses that co-sign, and addresses that sweep their whole balance to a common destination.

Applied to anonymity sets, this line of work has consistently found the delivered anonymity below the advertised figure. Kappos *et al.* (2018) reduced

the effective anonymity of the Zcash shielded pool using heuristics over its public usage patterns; Beres *et al.* (2021) profiled and deanonymised users of a fixed-denomination pool on Ethereum through address reuse and related behaviour.

What none of that work does is quantify the specific channel this article measures, namely the partition of a pool’s membership by the origin of each member’s funds, on Solana. The tracing rule used here follows each member’s funding chain to its birth edge, the oldest credit into the address, and stops at a terminal rule; Section 4.3 states the rules and the failure modes each one exists to avoid.

2.5 Positioning

Table 1 places this work against the four groups above along the axes that matter to its claims: what is measured, against which adversary, from which data, and whether the published figure carries an interval.

Table 1: Position of this work against the groups of published literature it builds on.

group	what it measures			adversary	data	interval
Entropy metrics (SERJANTOV; DANEZIS, 2003; DÍAZ <i>et al.</i> , 2003; ANDERSSON; LUNDIN, 2008; SMITH, 2009)	definitions of anonymity			stated ab- stractly	none, defini- tional	not appli- cable
Empirical anonymity-set studies (MÖSER <i>et al.</i> , 2018; KAPPOS <i>et al.</i> , 2018; BERES <i>et al.</i> , 2021)	advertised anonymity	against	effective	passive, chain- only	real, other ledgers	partial
Chain analysis (MEIKLEJOHN <i>et al.</i> , 2013)	address clustering and attribution			passive, chain- only	real	not appli- cable
Fixed-denomination pools (BEN-SASSON <i>et al.</i> , 2014; PERTSEV; SEMENOV; STORM, 2019)	unlinkability of a withdrawal from a deposit			passive, chain- only	design, not mea- sured	not appli- cable
This work	funding-provenance partition of a live pool, and the crowd a protocol can actually assemble			passive, chain- only, stated in full in Sec- tion 4.1	real, Solana main- net	two, kept apart

Three things distinguish the work, and each is a property of what it does rather than a judgement of the groups above. It acts on the provenance channel instead of only reporting a nominal k , by closing the action side of the problem in a protocol (Section 3). It measures the channel it cannot close against live data with a published method, rather than modelling it (Section 4). And it reports two distinct intervals with every figure, one bounding what the unresolved members could have been and one bounding the sampling error, because collapsing them produces a number that reads as more certain than it is (Section 4.2).

3 Protocol Design

This section presents the protocol. Section 3.1 gives the note model and the invariant that governs the escrow, Section 3.2 the circuit and the binding that ties a proof to one specific action, Section 3.3 the four instructions and the reason settlement is split into two phases, Section 3.4 the three shapes an action can take, and Section 3.5 the things the design refuses to build and why. The measured cost of everything described here is reported in Section 5.2 and Section 5.3.

3.1 The note model and the accounting invariant

A member’s note is a pair of secrets (k, r) held locally and never transmitted. Two values are derived from it: the commitment $H_3(k, r, d)$, where d is a tag for the pool’s denomination, which becomes a leaf of an append-only Merkle accumulator; and the nullifier $H_1(k)$, which is revealed when the note is spent and recorded by the program so that the note cannot pay out twice. The hash is Poseidon (GRASSI *et al.*, 2021), chosen because it is cheap to evaluate inside a constraint system.

The instances of the hash used for the three purposes differ in arity, and that difference is itself the domain separation: a Poseidon instance of width one, two or three is a different permutation, so a nullifier, a Merkle node and a commitment cannot collide by construction. An earlier design separated the domains with a small integer tag instead, which was wrong in a way worth recording: with a small tag constant, a Merkle node whose left child happens to equal the tag collides with a nullifier. Reaching that state requires a Poseidon preimage, so it was not exploitable, but arity separation removes the question rather than bounding it.

The denomination is a constant of the pool and not a field of the note. One pool serves one denomination, so the amount escrowed and the amount owed cannot disagree. This is what makes the first failure mode of Section 2.2 unrepresentable rather than merely untested: there is no amount in the note to bind to the commitment, so the bug in which a depositor of one unit withdraws the whole pool cannot be written. The second failure mode is closed by the nullifier set being global and never scoped to an epoch, so that no boundary can reopen a spend.

What the escrow must satisfy is then a statement about two counters:

$$\text{vault} \geq \text{denomination} \times (\text{deposits} - \text{settled spends}) + \text{rent} \quad (3)$$

Nothing a prover supplies appears in it. The invariant is re-read from the vault after lamports move rather than inferred from the arithmetic that

moved them, and the escrow lives in a dedicated account holding no data, so the balance it is checked against contains nothing but escrow and its own rent.

3.2 The circuit and the action binding

The statement proved is: *I know (k, r, d) such that $H_3(k, r, d)$ is a leaf of the tree with root R , my nullifier is $H_1(k)$, and this proof is bound to this action.* The proving system is Groth16 (GROTH, 2016), verified on chain through the BN254 syscalls of Section 2.1. The accumulator is 20 levels deep, so a pool holds up to 1,048,576 notes, and a membership path is 20 hashes whatever the occupancy.

Figure 1 shows which values are secret and which are public.

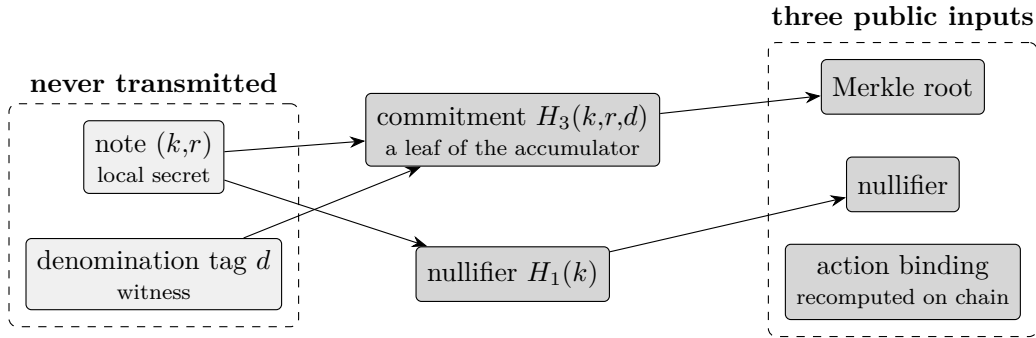


Figure 1: The note, the values derived from it, and the three public inputs of the proof. The action binding is recomputed on chain from the action about to execute and is never transmitted.

Source: prepared by the author.

Three values are public, and each is public for a reason that would break if it were a witness. The root must be checked by the program against its own history of the last 128 roots, so a proof stays valid across 128 subsequent deposits and no longer. The nullifier must be recorded to prevent replay. The action binding must be recomputed by the program from the action it is about to execute. The denomination tag stays a witness because membership already constrains it: a pool's tree contains only leaves committed at that pool's denomination.

The binding is a keccak digest, under a domain tag, over the selector, the target program, the beneficiary, the relay, the relay fee, the declared account count and the payload. It is never transmitted. The program recomputes it

from the action in front of it and uses the result as the third public input, so an intermediary that alters any covered field produces a different binding and the pairing fails. There is no separate transmitted field that could be compared incorrectly.

One subtlety decides whether any of this holds. A public input participates in Groth16 verification only through the constraint columns that reference it; an input that appears in no constraint has an all-zero column, its contribution is the identity, and every value satisfies the equation. The binding is therefore squared under constraint, which costs one multiplication and is the difference between a binding and a decoration.

Three implementations of the hash must agree: the constraint-system gadget, the host that builds the tree, and the syscall the program calls. The gadget and the host are each pinned against the published test vector of circomlib (IDEN3, 2026) rather than against each other, so that agreeing on a wrong answer would require that published vector to be wrong, and the syscall is then checked against the host on chain by asserting that the root the deployed program builds equals the root the host built. A gadget whose in-circuit and native hashes are different functions is the characteristic expensive failure of this kind of system, because nothing detects it until proving time and the symptom, proofs that verify nowhere, implicates everything except the hash.

The verifying key is derived from a public seed, so any reader can regenerate the key the deployed program checks against and compare it element by element. The comparison covers every element of the key rather than the one element a ceremony contribution would change, because a check confined to that element would certify a key belonging to an entirely different circuit. Section 6.2 states plainly what this property is not.

3.3 Four instructions, two phases

Figure 2 shows the member’s path.

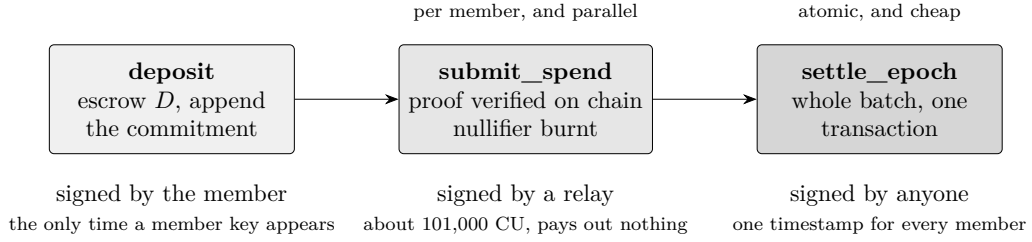


Figure 2: The four instructions, what each one costs, and which key signs it. No member key appears on chain after the deposit.

Source: prepared by the author.

`init_pool` creates the pool for one denomination and is permissionless. There is one pool per denomination globally, because splitting deposits of the same size across pools splits the anonymity set, and there is no privileged authority, so no key whose loss freezes the escrow.

`deposit` escrows exactly the pool’s denomination, read from the pool rather than from the instruction, and appends the commitment to the accumulator. The program stores only the accumulator’s frontier, which suffices to append a leaf and produce a root and does not suffice to prove that any particular leaf is present. Recovering the leaf set is therefore the client’s task, done from transaction history and checked by rebuilding the root and comparing it with the one the program holds. A matching root proves the recovered set is complete and correctly ordered, which is the precondition a membership proof needs, and it means a member can act from a machine that has never seen the pool.

`submit_spend` verifies the proof on chain, burns the nullifier, records the authorised action and pays out nothing. It is signed by a relay and never by the member, so no member key appears on chain after the deposit. The relay is read from the account that signed rather than from anything the caller states, which is what makes a proof spendable only by the relay it was made for.

`settle_epoch` executes the whole batch in one transaction, so every member’s action lands at one timestamp, in one ordering, under one signature. It is permissionless.

The split into two phases is a cost decision. Verifying n proofs inside one settlement would cost n times the cost of a verification, which exceeds the budget of a single instruction at six members (Section 5.2). Splitting them makes the expensive step per-member and parallel, and leaves the step that must be atomic, the one that gives every member the same timestamp, cheap.

Settlement applies the crowd rule, which is a disjunction: a batch settles

if it carries `k_floor` spends, or if every spend in it has waited out one hour. Requiring the crowd unconditionally would be a liveness hazard, since a quiet pool could hold a member’s escrow until a crowd that never arrives; dropping it would make synchronisation a word rather than a property. This is precisely the threshold-or-timed batching that Serjantov, Dingle and Syverson (2002) analyse, and Section 6.2 states what their result implies here.

3.4 Three shapes of action

An action is selected by a selector that is itself covered by the binding, so a settler cannot revise the member’s choice.

Table 2: The three action shapes. The selector is covered by the action binding, so the choice belongs to the member and settlement cannot change it.

selector	behaviour	what it is for
0	pay the beneficiary, no cross-program invocation	the degenerate case, kept because expressing “pay this account” should not require a target program
1	fund the beneficiary, then invoke the target	actions whose target must see the value before it acts
2	invoke the target with the pool’s vault as a signer, then pay	actions that need an authority rather than a funder, such as a delegation of stake or a governance vote

Selector two is the reason the protocol is a behavioural tool rather than a mixer with an invocation attached. A delegation of stake requires the staker authority to sign, and a member who signs is a member who has appeared on chain, so the only candidate is the pool.

The split between selectors one and two is forced by the runtime and was found by measurement rather than by reasoning. The program moves the vault’s lamports by direct mutation; an account mutated that way and then handed across an invocation boundary makes the runtime reject the entire instruction as unbalanced. Paying before the call and signing the call are therefore mutually exclusive, and which one an action needs is a property of that action, so the member chooses.

The constraint turned out to be a property of the batch rather than of the spend, and only a live cluster revealed it: the runtime objects to any lamport the program moved anywhere in the same instruction, so a signed call settled behind other members’ payouts fails where the same call alone succeeds. Settlement therefore runs every signed call before any payout. The shape of that fix matters on its own: a signed action that could only settle alone

would have to wait out the timeout instead of joining a batch, surrendering the shared timestamp that is the reason to stand in a crowd at all.

3.5 What the design refuses to build

Four omissions are deliberate, and each is a decision rather than an unfinished feature.

A batch of unequal fees is refused outright. A member receives the denomination less the relay fee, and that figure lands in a public account balance, so a batch whose members paid different fees settles into visibly different payouts and an observer partitions it by value. No proof is broken and no secret is learned: the balances are simply different, and the crowd rule, the shared timestamp and the single signature are all defeated by arithmetic. The program therefore rejects such a batch with a dedicated error. The check belongs to settlement rather than to submission because it is a property of the batch and not of any record.

There is no escape-hatch instruction, because none is needed. A member may act as their own relay at a fee of zero and settle their own batch once the timeout has passed, so nothing in the protocol can hold their escrow. The cost is the expected one, namely that their wallet signs and they give up the anonymity the relay path provides.

There is no auditor role and no viewing key. A member who must show one counterparty that a settled action was theirs produces a disclosure file for that one verifier, who checks it by recomputing every value in it: the commitment and the nullifier from the member's secrets, the record's address from the recomputed nullifier, the accumulator from chain history checked against the pool's own root, and the action read out of the record. A standing capability held by somebody else is a path that can be compelled; a file handed to a chosen verifier is not. Two guards apply. The command refuses before settlement, because until the nullifier is burnt the secrets are the deposit and anyone holding them can redirect the payout. It also refuses when the disclosure would push the remaining set below the pool's floor, since naming one action as yours removes you as a candidate for every other action in the pool, and that cost is paid by the other members.

There is no reward for holding a note longer than needed. It was designed and then removed, together with the entry fee that would have funded it, because a fee with no payout path is a trap for funds rather than a half-built feature. The deeper reason it stayed removed is that a reward must be paid to somebody, and naming a member's address on chain costs that member all of their anonymity at the exact moment they are being rewarded for protecting everyone else's. The anonymity-preserving form, a

second nullifier claimed under the same floor and paid to a fresh address, is specified rather than sketched, and is absent rather than half-present.

4 Measuring the Funding-Provenance Channel

The protocol of Section 3 closes the action side of the problem and leaves the membership side open, for the reason given in Section 1: no pool controls where the capital of its users came from. This section specifies how that channel is measured. Section 4.1 states the adversary, Section 4.2 the metric and what is published beside it, Section 4.3 the sampling frame and the tracing rules, and Section 4.4 the failure accounting and the gates that refuse to publish. The results of applying it are in Section 5.4 to Section 5.7.

An instrument of this kind is only as good as its weakest rule, and none of those rules is visible in its output. Each one below is therefore given with the specific failure it exists to avoid.

4.1 The adversary

Every figure reported here is meaningless without this paragraph beside it. The adversary is passive, global, retrospective and confined to the chain. They read the finalised ledger up to a pinned slot, run any analysis they like over it, and never compromise a key or a machine. They know the full membership of the measured set, because a deposit is a public event. They are granted the labelling function of this work exactly, no more.

What they are denied is equally part of the definition: off-chain identity, whether from exchange records or legal compulsion; observation at the network layer; correlation of timing or of amount between a deposit and an action; behavioural fingerprints such as wallet software or fee patterns; and the ability to make deposits of their own into the measured pool. Those are separate channels. Folding any of them in would inflate the figure and destroy its comparability with results measured under the stated adversary.

One consequence follows and is stated in Section 6.3 as a non-claim: a real adversary holds some of the denied channels, so the anonymity a real member enjoys is **below** what is reported here, never above.

The labelling is subject to one rule that the rest of the method is built around. A partition is admissible only if the adversary can compute the same label on the member side and on the action side. Entropy over a partition decreases monotonically under refinement, so finer labels always yield a lower figure, and a weaker tracer that fails to merge two members reports a worse

number than a stronger one. A metric with that property measures the tracer’s budget rather than the pool. Labelling members by raw address violates the rule, because “funded by this particular exchange wallet” is not observable of an action whose trace reaches a different wallet of the same exchange.

4.2 The metric and what is published with it

The headline is the loss factor $\rho = 2^{-H(C)}$ of Section 2.3, reported because it is independent of K and therefore comparable across pools of different sizes. An effective- k measured at small K understates the steady-state loss and cannot be extrapolated upward.

A point estimate alone is not publishable output here, and two intervals are reported with it because they answer different questions and neither covers the other.

The **unresolved bracket** is an exact bound. Members whose funding chain terminates without reaching a class are not a class. Merging them into one assumes they are all alike, which flatters the pool and raises the figure; splitting them into singletons assumes they are all distinct, which flatters the adversary and lowers it. Neither is known, so both extremes are computed and the interval between them is published.

The **sampling interval** is a percentile bootstrap over members, resampled with replacement at a published seed with a generator implemented in the analysis itself, so that a reader recomputes this interval rather than a similar one. It answers how much of the figure is an artefact of which depositors were drawn, which the bracket says nothing about.

Two further quantities are published with every run, not as results but as diagnostics of how much of the distribution was seen: Good–Turing coverage and the Chao1 richness estimate of Section 2.3. Section 5.4 shows why they earn their place.

One bias is not corrected and is stated because it runs against the argument of this work. The plug-in estimator of $H(C)$ is biased downward when classes are many and members few, which is the regime of every run reported here, and since $\rho = 2^{-H(C)}$, **every ρ published here is biased upward: the pools plausibly leak less than reported.** A bootstrap resamples the same estimator, so its interval is centred on the biased value and does not repair this. The bias falls in the same direction on every population measured the same way, which is what keeps a difference between two populations meaningful where an absolute figure is fragile.

4.3 The sampling frame and the tracing rules

Figure 3 shows the two passes and the point at which each gate applies.

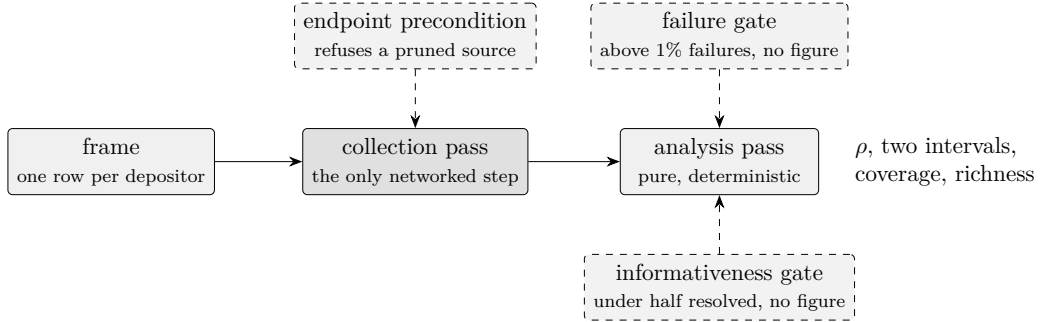


Figure 3: The measurement pipeline. Only the collection pass touches the network; the analysis pass is pure, deterministic and reproducible offline from the recorded sample.

Source: prepared by the author.

The frame is member-weighted: one row per depositor, enumerated from the pool’s own deposit transactions. Sampling addresses because they appear in recent blocks is weighted by transaction volume instead, which biases the frame toward the highest-frequency actors, whose funding histories are also the most expensive to walk. Section 5.4 reports what that error costs in practice, because it was made and measured before it was corrected.

Nothing is excluded from the frame on a criterion correlated with the outcome. Discarding wallets with long histories because they are expensive to trace would drop exactly the wallets with deep, traceable funding, leaving a remainder that is disproportionately untraceable; the resulting headline would measure the exclusion rule.

Tracing proceeds in two passes, and the split is the point. The **collection pass** walks each member’s funding chain and writes everything it observed to a sample file. It is the only step that touches the network. The **analysis pass** classifies over the complete sample, offline. Two of the terminal rules are properties of the set rather than of an individual address, so classifying during traversal would make a member’s class depend on visit order. Given the same sample, the analysis pass always produces the same partition, which is what allows a published figure to be checked by a third party with no network access at all.

Four rules govern the walk, and each exists to avoid a specific and silent failure.

Edges come from balance deltas, not from parsing instructions.

An extractor that recognises transfer instructions is blind to every program that moves value by mutating an account directly, and to account creation and closure.

The birth edge is the oldest credit into an address, so the walk reaches the beginning of a wallet’s history. Reading the most recent transactions instead is the wrong end of the record for any wallet with more than a handful of them, and it fails silently: the output is “unresolved”, which is indistinguishable from a genuinely unresolvable wallet.

The oldest transaction is where that search begins, not where it ends. An address routinely appears in somebody else’s transaction before it is ever funded, in which case its oldest transaction credits it nothing. Treating that as the absence of funding converts *the tracer stopped reading* into *there is nothing there*. Section 5.5 reports what this cost when it happened here.

The threshold that makes an address a hub is decoupled from the paging cap. Collapsing them into one number is easy, because both answer the question of how many signatures to look at, and it quietly turns “reaches an attributable origin” into “hit the paging cap”, which admits every high-volume program and trading bot as an origin while excluding a genuine exchange withdrawal address of moderate activity.

The terminal rules themselves are the Solana analogues of the clustering heuristics of Section 2.4 (MEIKLEJOHN *et al.*, 2013), together with a definitional test that separates an address owned by a program from an address owned by a person.

4.4 Failure accounting and the gates

The governing principle is that an infrastructure limit must never read as an absence of evidence. Three outcomes are therefore counted separately and never merged.

- **Evidence-unresolved:** the chain genuinely offers no qualifying incoming edge. A claim about the ledger.
- **Budget-unresolved:** the walk exhausted its depth or its paging allowance. A claim about the instrument.
- **Infrastructure failure:** a request failed. Not evidence of anything, excluded from the class distribution entirely, and reported on its own line.

The idiomatic way to keep a traversal running past an error is to treat a failed lookup as an empty result, and doing so is silently catastrophic: it launders an infrastructure failure into a data point, in the direction that

inflates the unresolved bucket, and the output is indistinguishable from a genuinely hard-to-trace population.

Counting failures and printing a warning is not sufficient, because a headline printed above a warning is still a headline. Three gates therefore refuse rather than annotate.

1. **The failure gate.** Above a failure rate of one per cent, the run prints its census and refuses to print a figure.
2. **The informativeness gate.** If fewer than half the attempted members reach a class, no figure is published, because what survives after the unresolved are dropped is the population’s traceable subset rather than the population.
3. **The endpoint precondition.** A data source with pruned history returns an empty result rather than an error for old events, so a collection against one looks healthy while reporting every old funding event as absent. The endpoint is therefore probed for archival depth before a run starts, and a run against one that fails the probe does not begin.

A fourth check tests the assumption that the other three rest on. Every figure computed from the resolved members alone assumes those members are a fair draw of the classes present. That is testable: collect one frame at two budgets, split the resolved members into those the smaller budget also reached and those only the larger one did, and bootstrap the difference in ρ between the two groups. **Separation is the bad outcome.** It would mean that resolvability correlates with provenance class, that the resolved subset is biased toward whatever is cheap to trace, and that no additional budget repairs it. Section 5.6 reports the result on both populations.

5 Evaluation

This section evaluates both halves of the work. Section 5.1 states where each figure was taken. Section 5.2 and Section 5.3 answer the first sub-question of Section 1, on the cost of a proof and the size of a crowd. Section 5.4 to Section 5.7 answer the second and third, on how much of the nominal anonymity survives the funding-provenance channel of Section 4.

5.1 Where each figure comes from

Three environments are used and every figure below belongs to exactly one of them. **A local virtual machine** loads the compiled program and executes real transactions against it, including real proof verification through the actual syscall; it is where the byte and compute measurements of Section 5.2 and

Section 5.3 are taken, by serialising the real instruction and counting what it names. **A live devnet cluster** carries the deployed program and is where the whole lifecycle was run end to end, so that a figure taken locally can be confirmed against a real validator. **Solana mainnet, read only**, is the source of the provenance data of Section 5.4; nothing in this work writes to mainnet, and Section 6.2 explains why the protocol is not deployed there.

Two figures in Section 5.3 are computed rather than landed, and they are marked as such in the same sentence that reports them.

The complete implementation, the recorded samples behind every provenance figure, and the analysis that reproduces them are available in the project’s public repository¹, on the branch `feat/provenance-aware-pool`. Reproducibility here has a specific meaning rather than a general one: the analysis pass is pure and deterministic, so it recomputes a published figure from the recorded sample with no network access; its bootstrap generator is implemented in the analysis itself and seeded from a published constant, so a reader recomputes this work’s interval and not merely a similar one; and the samples of runs that produced no publishable figure are kept alongside those that did, because a failed run is evidence about the method.

5.2 The cost of a proof and of a settlement

On-chain verification measures as $74,179 + 5,661 \times N$ compute units for N public inputs, so each public input costs about 5,661 CU. At the three public inputs of Section 3.2 the verification is 91,162 CU. The whole spend instruction, which also burns the nullifier and records the action, costs about 101,000 CU, roughly half the 200,000 CU a single instruction is given by default. The two figures are different quantities and are not interchangeable.

That difference is what forces the two-phase design of Section 3.3. Verifying six proofs inside one settlement would cost on the order of 606,000 CU, three times the default budget of an instruction and a large fraction of the maximum a transaction may request at all.

¹[<https://github.com/thiagorochatr/mirror-pool>](https://github.com/thiagorochatr/mirror-pool)

Table 3: Compute cost of settlement by shape of batch. Every row but the last was executed; the last is computed from the per-member rate the row above it measured.

batch	compute
10 plain payments	19,545 CU
20 payouts settled through a lookup table	35,895 CU
4 cross-program actions and their payouts	40,251 CU
1 action signed by the pool’s vault	under 50,000 CU
6 delegations of stake to 6 different validators, on devnet	142,856 of 200,000 CU
13 delegations of stake, computed at the measured per-member rate	about 309,400 CU

The figure for four cross-program actions moves by a few thousand units between runs, because the accounts are generated afresh each time and address derivation searches a different number of candidate values. What is asserted is therefore the property, that four such actions and their payouts stay under 60,000 CU, rather than the number itself.

The last row is the first place where compute becomes the binding constraint rather than a comfortable margin, and Section 5.3 shows what happens when it does.

5.3 How large a crowd fits in one settlement

What this number is not. It is not the size of the anonymity set. The set is the accumulator, which holds up to 1,048,576 notes, and a proof asserts membership of it without naming a leaf, at the same cost whatever the occupancy. What a settlement bounds is narrower: how many members share *one timestamp*. A pool with more members than one batch holds settles in several batches, paying for it in timestamps rather than in set size.

Under a legacy transaction, every account is named by its full 32 bytes and the 1232-byte packet binds first. Through an address lookup table, each account costs one byte and the packet stops binding; what takes over is the limit on account locks.

Table 4: Members per settlement under the two transaction formats. The legacy column and the first row of the lookup-table column were executed; the two lookup-table figures for delegations are computed by building the real instruction and counting the accounts it names, against a lock limit that was itself measured on devnet.

batch	members, legacy	bytes	members, through a lookup table
plain payments	10	1228	20
delegations of stake, all to the same validator	7	1140	18
delegations of stake, a different validator each	6	1194	13

The legacy ceilings were taken by measurement and confirmed from both sides. The eleventh payment was refused by the wire at 1327 bytes, 95 over the limit, while the same batch settled in the local virtual machine in 24,158 CU: compute is demonstrably not the constraint for payments, and the test that establishes this replays the identical batch into a second pool so that the reason is checked rather than assumed. The six-member delegation batch weighs 1194 bytes both in the local measurement and in the transaction that landed on devnet, which is what makes the ceiling a prediction rather than a description.

Through a lookup table, twenty payouts settled on devnet in 332 bytes of the 1232 available, with a **single required signature**, naming two accounts statically and resolving 62 through the table, at 35,895 CU. Twenty recipients and twenty relays are named in that transaction and not one of them signed it. The same batch as a legacy transaction would weigh 2218 bytes, 986 over the limit.

What binds instead is the limit on account locks, and that figure was taken the hard way: a batch of 24 members was refused by devnet at 77 accounts. The raised limit of 128 exported by the current transaction library is not live on that cluster, so a client must plan against 64 until it can observe otherwise. The arithmetic is exact at twenty members: three accounts per member, namely the spend record, the beneficiary and the relay, plus four the batch names once, namely the pool, the vault, the settler and the program itself, giving 64. The transaction above therefore sits precisely on the limit.

Two consequences are worth stating as results rather than as inconveniences.

Divergent behaviour costs anonymity-set size. A member who chooses their own validator names a vote account no one else in the batch names, so the batch loses a member: seven become six under the packet,

and eighteen become thirteen under locks. The gap widens from one member to five, because a shared vote account is named once either way but is also *locked* only once, so agreeing on a validator buys more under locks than it did under bytes. This is a real trade-off in the design and not a defect of the measurement: the behaviour a member most wants to hide is exactly the behaviour that shrinks the crowd hiding it.

Escaping one limit is paid for out of the other. The usual answer to a settlement that runs out of compute is to request more with an explicit compute-budget instruction. Measured against the six-member delegation batch, that instruction costs 40 bytes, and a full legacy settlement has 38 to spare. In a legacy transaction, raising the budget therefore means dropping a member. A lookup table lifts the byte constraint, but the compute-budget program is itself an account, so raising the budget still costs a lock. The two figures for delegations in Table 4 include that cost, because at thirteen members the request is not optional: the projected 309,400 CU of Table 3 is past the default budget.

Deployment evidence. The whole lifecycle ran on devnet against a real validator: pool creation, deposits, spends each carrying a proof verified by the deployed program’s own syscall, and a settlement that closed the vault to its rent-exempt minimum to the lamport, with 100,000,115 lamports owed against 100,000,115 paid out and a remainder of zero. That settlement carried five spends, and two of them were not transfers. One was a memo signed by the pool. The other was a real delegation of stake, with the pool’s vault as the staker authority and the operator as the withdraw authority. The withdraw authority is deliberately not the pool, which is Section 6.2 applied rather than restated: the pool’s signature is available to every member, so an authority the vault holds is an authority every member holds. A delegation survives that property, because the worst a member can do with it is re-delegate somebody’s stake. A withdrawal would not.

5.4 The provenance measurement: eight runs

The population measured is not this project’s pool. It is a live and unrelated Solana privacy pool, identified by its on-chain program address `9fhQBbumKEFuXtMBDw8AaQyAjCorLGJQiS3skWZdQyQD`, because the pool of Section 3 has no depositors and measuring an empty pool measures nothing. The control population is the depositors of a liquid staking protocol, `MarBmsSgKXdrN1egZf5sqe1TMai9K1rChYNDJgj7aD`, chosen because staking is an ordinary and publicly attributable act that nobody performs for deniability.

The headline run resolved 54 of 83 attempted members with **zero infras-**

tructure failures and zero members reported as having no funding at all, giving

```
rho = 0.0955    unresolved bracket 0.0350 ... 0.1136
               sampling interval  0.0848 ... 0.1790
```

over 23 observed classes. Knowing a member's funding class costs that pool roughly an order of magnitude of its nominal anonymity.

Table 5: All eight collection runs. Two produced a published figure and six did not: run 1 resolved nothing, run 2 failed the failure gate, runs 3, 7 and 8 failed the informativeness gate, and run 5 was invalidated by the defect of Section 5.5. The attempted count for run 5 was not recorded separately before that run was superseded.

run	population	budget	attempted	resolved	outcome
1	addresses drawn from one block	transaction-weighted frame	12	0	no figure: 11 of 12 seeds hit the paging cap at 20,000 lifetime transactions
2	privacy pool	depth 8, page cap 60	87	47	no figure: failure rate 14.94%, above the one per cent gate
3	privacy pool	depth 8, page cap 8	84	40	no figure: 40 resolved against a threshold of 42. ρ 0.1219, bracket 0.0253 to 0.1837
4	privacy pool	depth 16, page cap 24	84	50	first published figure. ρ 0.1032, bracket 0.0316 to 0.1318
5	staking control	depth 16, page cap 24	—	16	invalidated by the defect of Section 5.5
6	privacy pool	as run 4, defect corrected	83	54	the headline. ρ 0.0955, bracket 0.0350 to 0.1136
7	staking control	as run 5, defect corrected	92	38	refused: 38 of 92 is under the informativeness gate
8	staking control	page cap doubled to 48	92	42	refused: 42 against the 46 required

Runs 4 to 8 had their budget and their prediction committed before collection began, so the parameters are declarations rather than descriptions and the predictions are on the record, including the one that turned out wrong (Section 5.6).

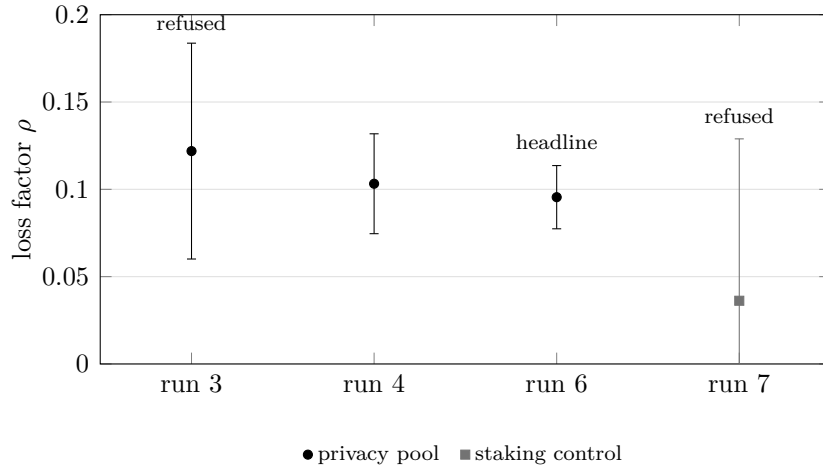


Figure 4: The loss factor and its unresolved bracket, for the four runs that published one. Runs the gates refused are marked; the width of a bracket, not the position of a point, is what the gates respond to.

Source: prepared by the author.

The tail is a finding rather than a caveat, and it is the most transferable result in this subsection. Run 4 resolved ten more members than run 3 and came back with *worse* coverage, 0.62 against 0.65, and a richness estimate that rose from 108 classes to 193. The additional members did not fall into the classes already observed; they fell into new ones, mostly alone. Under Good–Turing, coverage falls when the share of classes seen exactly once rises, so a sample that keeps discovering fresh singleton classes reports less confidence as it grows.

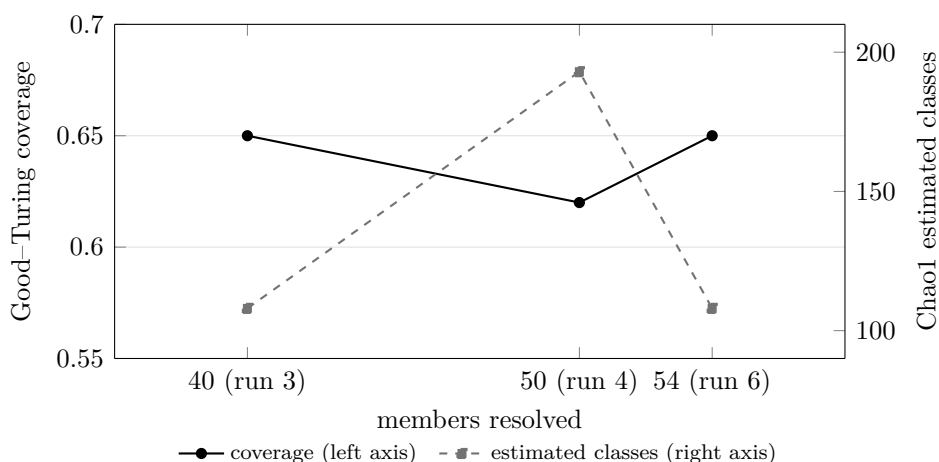


Figure 5: Coverage and estimated richness against the number of members resolved, over runs 3, 4 and 6. Resolving more members did not make the class distribution better observed.

Source: prepared by the author.

The consequence changes what the headline means. Under-sampling here is a property of the provenance distribution and not an artefact of the budget: there is no budget at which this frame’s class distribution becomes well observed, because the mass is in the tail. The honest statement of what was measured is therefore that ρ is about 0.10 on this pool, inside a bracket from 0.0350 to 0.1136, from a sample whose class distribution is demonstrably heavy-tailed and mostly unobserved, with the second fact reported as a finding.

5.5 A defect in the instrument, and what it cost

Run 5 produced an unresolved bucket six times larger than the privacy pool’s, and in the evidence category rather than the budget one. Checking two of those members by hand found the cause, and it was in the instrument: the tracer implemented *the oldest transaction, if it is a credit* where the documented rule of Section 4.3 is *the oldest credit*. An address whose first appearance is in somebody else’s transaction, which is common, was therefore reported as having no funding at all.

Every figure published before that point, including the run 4 headline, was computed with the defective tracer, so both populations were collected again.

Table 6: Why each chain stopped, before and after the correction, on the privacy pool. The second row is the defect: ten claims about the ledger that were really claims about the tracer.

stop reason	run 4 (defective)	run 6 (corrected)
paging cap reached	69	76
no incoming edge	10	0
local terminal	4	5
birth scan exhausted	not present	1
depth exceeded	1	1

The evidence bucket went to zero, and that is the result. Ten addresses had been reported as having no funding credit whatsoever, which is an assertion about the chain rather than about the instrument, and every one of them was wrong. On the control the defect bit more than twice as hard, with 36 of 92 members wrongly reported as unfunded, which is what made it visible at all. One address now reports honestly that the walk stopped before finding its credit, which is a claim about the instrument and is counted as budget.

The corrected headline moved down, from 0.1032 to 0.0955, and the bracket narrowed by roughly a third. The correction happened to move the figure in the direction that flatters this work; it was made because the code disagreed with its own specification, and the direction was not known until the run finished.

5.6 Selection, and the comparison that was refused

The check specified in Section 4.4 asks whether the members that resolve are a fair draw of the classes present.

Table 7: The selection check on both populations. Neither separates, so at these margins being resolvable does not pick out particular provenance classes.

population	pair of budgets	cheap to trace	expensive to trace	difference, 95%
privacy pool	depth 8 and cap 8, against depth 16 and cap 24	39 members, ρ 0.1179	15 members, ρ 0.1250	-0.1507 to +0.0704
staking control	identical budgets, defect corrected	16 members, ρ 0.0743	22 members, ρ 0.0585	-0.0144 to +0.0786

Neither interval excludes zero, which is the good outcome. **Only the first row is a genuine budget margin**, and the second is weaker than it looks: the two control runs used identical parameters and differ by the correction of Section 5.5, so its expensive group is “members the defective tracer failed on” rather than “members a smaller budget could not reach”. It is a real check on whether that defect selected for particular classes, which is worth knowing, and it is not a second budget margin.

The comparison between the two populations is the third sub-question of Section 1, and it is reported as **unachieved**.

population	members	rho	resampled 2.5-97.5%	bias
privacy pool	54	0.0955	0.0848 ... 0.1790	+0.0276
staking control	42	0.0335	0.0428 ... 0.0657	+0.0188

difference (privacy pool - staking control): +0.0619, 95% +0.0307 ... +0.1267

The interval on the difference excludes zero and points the way this work’s argument would prefer: the privacy pool reads as the more provenance-concentrated population. The difference and its interval are resampled directly rather than obtained by subtracting the two point estimates, which is why it does not equal their difference to the last digit.

The tool refuses to report it, because the control resolves fewer than half of its members and the informativeness gate of Section 4.4 fires. The reasoning is not a formality: what survives after the unresolved are dropped is each population’s traceable subset, and traceability is not independent of provenance class, so two traceable subsets can differ in concentration purely because their *unresolved* members differed, with nothing in the numbers to reveal it.

That gate did not exist when the comparison was first implemented; it was added after the control was seen to fall on the wrong side of it. Stated plainly: **the check that refuses this work’s own favourable result was built after the result was known to be favourable**.

Run 8 then spent a doubled budget attempting to clear the gate, having predicted in advance, and in the knowledge of which outcome would suit the argument, that it would. **It did not**. Resolution rose from 38 to 42 against the 46 required, and the prediction is on the record as wrong. The stopping rule had been fixed before collection and there was no ninth run.

The most informative number in run 8 is not ρ . Doubling the paging cap bought four additional resolved members for 1,246 further requests, which is 311 requests per additional member against about 50 for that population as a whole. The control is not under-resolved through parsimony: the members still missing have genuinely long funding chains, and each further one costs more than the last. The staking pool, where nobody seeks deniability, is

therefore markedly harder to trace than the privacy pool, which resolved 54 of 83 members at about 33 requests each. Two explanations cannot be separated with this data, namely that staking attracts older wallets with deeper histories, or that privacy-pool depositors disproportionately fund from exchanges, which terminate in one hop. The second would sharpen the concentration finding rather than soften it, which is precisely why it is not asserted.

5.7 The metric applied to this work's own pool

It would be easy to publish Section 5.3 and let a reader take its figures for a privacy result. They are not one, and the way to show that is to run the measurement against this work's own crowd rather than only against somebody else's pool.

On the six-member devnet crowd of Section 5.3, the metric returns $\rho = 1.0000$, the best value it can produce, over a single class. **The figure is worthless, and the reason is instructive.** Every note in that pool was funded by one wallet, so the partition has one class holding all six members; an adversary who learns a member's funding class learns nothing, and the metric correctly reports no loss through that channel. What it cannot report is that the single class is the operator, who funded every deposit and every relay and therefore knows exactly which member is which. Against that adversary the anonymity set is one, and no funding-provenance figure will ever say so, because provenance is not the channel that failed.

This is the tautology a metric of this kind invites, met directly: applied to a population constructed by the person reading the result, it returns whatever that construction implies. The published headline avoids it only because it is pointed at a pool this work neither controls nor funded.

6 Discussion, Limitations and Non-Claims

Section 6.1 states what the two halves of the work buy together, Section 6.2 the limitations and threats to validity, and Section 6.3 what the work does not claim. The three are of equal standing: a result of this kind is only worth what its stated limits are worth.

6.1 What the two halves buy together

The protocol of Section 3 closes the action side of the problem. Actions execute from the pool's vault, so the on-chain trace of a delegation made through the pool leads to the pool and is identical whichever member asked

for it, and settlement gives every member of a batch one timestamp, one ordering and one signature. Against the adversary of Section 4.1, arrival time cannot separate members whom the proof has already made indistinguishable, because there is only one arrival.

The measurement of Section 4 quantifies the side the protocol cannot close. On the pool measured in Section 5.4, knowing a member's funding class costs roughly an order of magnitude of the nominal anonymity. That figure is not a property of this protocol, and it would be true of any deposit-based set on a public ledger.

The two together produce a claim narrower than either would alone. **A pool of this design does not deliver an anonymity of k ; it delivers the action side closed, and a membership side whose leakage is measured and published beside the number of members.** The alternative, and the common practice, is to report k and leave the channel unmeasured, which advertises an anonymity nobody has checked. The contribution of the second half is therefore not the value of ρ but the fact that it is measurable, expensive to measure honestly, and does not converge (Section 5.4).

There is a symmetry worth naming between the two halves. Section 5.3 found that the behaviour a member most wants to hide, the choice of a specific validator, is exactly the behaviour that shrinks the crowd available to hide it. Section 5.4 found that the members most worth resolving are the most expensive to resolve. In both cases the cost of privacy rises with the specificity of the thing being protected, from opposite directions.

6.2 Limitations and threats to validity

The trusted setup is reproducible and not secure. The seed is public, so the toxic waste is public, so proofs are forgeable by anyone who runs the setup. Reproducibility buys the ability to regenerate the verifying key and compare it element by element against the one the deployed program holds (Section 3.2), which is a real and lesser property. This is why the protocol is deployed to devnet and not to mainnet: a live pool with forgeable proofs would invite deposits it cannot protect, and publishing a threat model that says proofs are forgeable while advertising a mainnet address would be incoherent. Devnet exercises the same runtime, the same syscall and the same verifier; what mainnet would add is a claim about readiness that this setup does not support. The prerequisite is a multi-party ceremony, not more funding.

The action binding fixes how many accounts an action takes, not which ones. Settlement is permissionless, so a settler chooses them. For a target whose destination lives in instruction data this changes nothing; for a

target whose destination is an account, it is a real gap. A member's escrow is not at risk, because only this program can debit the vault, but anything the action itself would move is.

The submission phase is public and is the weaker half of the design. A spend is one transaction per member, at a moment of the relay's choosing, and it publishes in clear text everything about the action except who asked for it. Three consequences follow. A relay that submits on request leaks when the member asked, which is relay policy rather than a protocol guarantee. Settlement executes a batch in the order the records are passed, so a settler who preserves submission order makes position in the batch a restatement of arrival order. And a member who relays for themselves names themselves beside the action they are about to take. The last is not a subtle statistical channel but a single public transaction that ends the question, and the honest statement of what the protocol provides is therefore conditional: *given that the member never signs and never funds their own relay, the settled action cannot be attributed to them*. The first clause is a discipline the member keeps, not a property the program enforces.

The crowd rule has no floor on its timeout branch. A batch settles with `k_floor` spends or when every spend in it has waited an hour, and the second branch has no minimum size, so a batch of one settles and executes. Settlement being permissionless, an adversary may be the settler and choose the moment. This is the weakness that Serjantov, Dingledine e Syverson (2002) identify in the disjunction, and it is accepted deliberately: the alternative, an unconditional floor, makes a member's escrow hostage to the arrival of strangers, and a pool that never reaches its floor again would freeze every note in it permanently, with no authority able to release them. Given the choice between an action that may be attributable and money that may be unrecoverable, this design takes the first. What would repair it properly, and is not built, is a batch that refunds rather than executes when it fails to reach the floor.

The pool's signature is available to every member. That is safe only because the vault owns nothing but its own lamports; it stops being safe the moment the pool is made an authority over shared state, which is why the devnet delegation of Section 5.3 gives the vault the staker authority and never the withdraw authority. **The program is upgradeable**, which is a named trust assumption and not a property. **Amounts are public**, since denominations are pool constants, and there is no confidential-value layer. **The crowd floor bounds program-visible membership only**, which is all a program can check, so a pool whose deposits are largely one actor's has a large nominal k and a small real one. **The work is not audited**: the end-to-end suite is evidence of behaviour and not a substitute for external

review.

On the measurement side, four limits bound what Section 5.4 supports. **The scope is one channel, one adversary, one moment and one labelling**; a real adversary holds the channels denied in Section 4.1, so real anonymity is below what is reported. **The label ladder specified for the method was not fully built**: the implementation classifies once and reports one figure, so there is no bracket over label resolutions, and the bracket that is published answers a different question. **No step re-derives the curated entity labels from the chain**, so those labels rest on their cited sources. **The tracing scope is native value only**, and signature indexing does not name a recipient wallet for a plain token transfer into an existing associated token account, so a wallet funded in stablecoins, which is a common withdrawal path from exchanges, may have no history at its own address for the funding event. For such a wallet an “unresolved” outcome is an artefact rather than a finding, and no third-party empirical study was found that would let this work bound how often that occurs on Solana.

6.3 What this work does not claim

- **Not “anonymous”, “untraceable” or “unlinkable”** without a named adversary and a stated population. Every quantitative claim above names the environment it was measured in.
- **Not that the funding-provenance channel is closed**. It is closed on the action side and measured on the membership side, and those are different verbs.
- **Not that the trusted setup is secure**. It is reproducible, which is a different and lesser property.
- **Not that privacy pools attract more concentrated funding than ordinary users**. A control was measured to find out, the point estimates say they do, and the sample does not support saying it (Section 5.6). Comparability of the metric across populations is demonstrated as machinery and unproven as a finding.
- **Nothing about any individual member**. The metric is a property of a distribution, no member address appears in any published artefact, and the worst-case class of size one is a feature of heavy-tailed provenance in general rather than a finding about a person.

7 Conclusion and Future Work

This work set out to give an anonymity set to protocol actions rather than to funds, to find what bounds such a crowd in practice, and to measure how much of a nominal anonymity survives the funding-provenance channel. The five contributions announced in Section 1 are answered here in the order they were made.

The protocol delivers the action side. Members prove membership in zero knowledge, the pool executes and signs the action, and a batch settles under one signature at one timestamp (Section 3). The capability that distinguishes this from a value mixer was exercised on a live cluster: a real delegation of stake, signed by the pool as the staker authority, settled in the same transaction as three transfers and a memo (Section 5.3).

The binding constraint on a crowd is the transaction format, not the cryptography. A proof verifies on chain for about 101,000 CU, half the default budget of one instruction, while a settlement of ten payments costs 19,545 CU. What runs out first is packet space, and then account locks: ten members per legacy settlement of payments, twenty through a lookup table, and six when each member delegates to a validator of their own choosing. Two results came out of that (Section 5.3): divergent behaviour costs anonymity-set size, and escaping one format limit is paid for out of the other in both regimes.

The measurement instrument works and refuses. It reports a loss factor with two intervals that answer different questions, counts infrastructure failures apart from evidence, and holds three gates that decline to publish rather than publishing with a warning (Section 4).

Eight runs produced one headline and four refusals. On a live and unrelated privacy pool, $\rho = 0.0955$ inside an unresolved bracket of 0.0350 to 0.1136 and a sampling interval of 0.0848 to 0.1790: knowing a member's funding class costs that pool roughly an order of magnitude of its nominal anonymity. Two secondary results are more transferable than the headline. The class distribution does not become better observed as more members are resolved, so under-sampling is a property of provenance rather than of a budget (Section 5.4). And the population that seeks no deniability at all turned out to be markedly more expensive to trace than the one that does, at 311 requests per additional member against about 33 for the privacy pool (Section 5.6).

The comparison this work most wanted is reported as unachieved. The point estimates favour its argument and the interval on the difference excludes zero; the gate refuses it because the control resolves under half its members, and that gate was built after the favourable result was known (Section 5.6). A run intended to clear it failed against a prediction committed

in advance.

The non-claims are part of the result rather than a disclaimer attached to it (Section 6.3). The two that most constrain what the rest may be taken to mean are that the trusted setup used here is reproducible and not secure, which is why nothing is deployed to mainnet, and that the funding-provenance channel is not closed but measured. A reader who takes only the headline from this work and leaves those two behind has taken the wrong half.

Five directions follow directly, each from a limitation of Section 6.2 rather than from a wish list. A multi-party ceremony would replace a reproducible setup with a secure one and is the single prerequisite to a deployment that could accept real deposits. Committing the action's account list into the proof would close the gap between binding how many accounts an action takes and binding which. A batch that refunds rather than executes when it fails to reach the crowd floor would let a member trade the action instead of the anonymity, which is the honest repair to the timeout branch. Extending the tracing scope to token transfers would remove the artefact that a stablecoin-funded wallet is indistinguishable from an unresolvable one. And repeating the cross-population comparison with a budget large enough to clear the informativeness gate would settle the third sub-question, which this work leaves open on purpose rather than by omission.

The narrower conclusion is the one to carry away. An anonymity set on a public ledger can be given to behaviour, and the size of the crowd that results is set by transaction format rather than by proof cost. The channel that no such design closes can be measured against real data, and doing so honestly is expensive, does not converge, and will sometimes return a result the measurer would rather not publish.

References

ANDERSSON, C.; LUNDIN, R. On the fundamentals of anonymity metrics. In: **The Future of Identity in the Information Society**. Boston: Springer, 2008, (IFIP Advances in Information and Communication Technology, v. 262). p. 325–341. Disponível em: <https://doi.org/10.1007/978-0-387-79026-8_23>.

ANZA. **solana-bn254: alt_bn128 syscalls for Solana programs**. 2026. Documentação da crate, versão 3.2.1. Disponível em: <https://docs.rs/solana-bn254/latest/solana_bn254/>. Acesso em: 28 jul. 2026.

BARRETO, P. S. L. M.; NAEHRIG, M. Pairing-friendly elliptic curves of prime order. In: **Selected Areas in Cryptography (SAC 2005)**. Berlin, Heidelberg: Springer, 2006. (Lecture Notes in Computer Science, v. 3897), p. 319–331. Disponível em: <https://doi.org/10.1007/11693383_22>.

BEN-SASSON, E.; CHIESA, A.; GARMAN, C.; GREEN, M.; MIERS, I.; TROMER, E.; VIRZA, M. Zerocash: Decentralized anonymous payments from Bitcoin. In: **2014 IEEE Symposium on Security and Privacy**. IEEE, 2014. p. 459–474. Disponível em: <<https://doi.org/10.1109/SP.2014.36>>.

BERES, F.; SERES, I. A.; BENCZUR, A. A.; QUINTYNE-COLLINS, M. Blockchain is watching you: Profiling and deanonymizing Ethereum users. In: **2021 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPS)**. IEEE, 2021. p. 69–78. Disponível em: <<https://doi.org/10.1109/DAPPS52256.2021.00013>>.

CHAO, A. Nonparametric estimation of the number of classes in a population. **Scandinavian Journal of Statistics**, v. 11, n. 4, p. 265–270, 1984. Disponível em: <<https://sites.google.com/view/chao-lab-website/publication>>. Acesso em: 28 jul. 2026.

CHAUM, D. L. Untraceable electronic mail, return addresses, and digital pseudonyms. **Communications of the ACM**, v. 24, n. 2, p. 84–90, 1981. Disponível em: <<https://doi.org/10.1145/358549.358563>>.

DÍAZ, C.; SEYS, S.; CLAESSENS, J.; PRENEEL, B. Towards measuring anonymity. In: **Privacy Enhancing Technologies (PET 2002)**. Berlin, Heidelberg: Springer, 2003. (Lecture Notes in Computer Science, v. 2482), p. 54–68. Disponível em: <https://doi.org/10.1007/3-540-36467-6_5>.

EFRON, B.; TIBSHIRANI, R. J. **An Introduction to the Bootstrap**. Boston: Springer, 1993. Disponível em: <<https://doi.org/10.1007/978-1-4899-4541-9>>.

GOOD, I. J. The population frequencies of species and the estimation of population parameters. **Biometrika**, v. 40, n. 3-4, p. 237–264, 1953. Disponível em: <<https://doi.org/10.1093/biomet/40.3-4.237>>.

GRASSI, L.; KHOVRATOVICH, D.; RECHBERGER, C.; ROY, A.; SCHOFNEGGER, M. Poseidon: A new hash function for zero-knowledge proof systems. In: **30th USENIX Security Symposium (USENIX Security 21)**. USENIX Association, 2021. p. 519–535. Disponível em: <<https://www.usenix.org/conference/usenixsecurity21/presentation/grassi>>. Acesso em: 28 jul. 2026.

GROTH, J. On the size of pairing-based non-interactive arguments. In: **Advances in Cryptology (EUROCRYPT 2016)**. Berlin, Heidelberg: Springer, 2016. (Lecture Notes in Computer Science, v. 9666), p. 305–326. Disponível em: <https://doi.org/10.1007/978-3-662-49896-5_11>.

IDEN3. **circomlib: circuit library and published Poseidon test vectors**. 2026. Repositório de código. Disponível em: <<https://github.com/iden3/circomlib>>. Acesso em: 28 jul. 2026.

KAPPOS, G.; YOUSAF, H.; MALLER, M.; MEIKLEJOHN, S. An empirical analysis of anonymity in Zcash. In: **27th USENIX Security Symposium (USENIX Security 18)**. USENIX Association, 2018. p. 463–477. Disponível em: <<https://www.usenix.org/conference/usenixsecurity18/presentation/kappos>>. Acesso em: 28 jul. 2026.

MEIKLEJOHN, S.; POMAROLE, M.; JORDAN, G.; LEVCHENKO, K.; MCCOY, D.; VOELKER, G. M.; SAVAGE, S. A fistful of bitcoins: Characterizing payments among men with no names. In: **Proceedings of the 2013 Conference on Internet Measurement Conference (IMC '13)**. ACM, 2013. p. 127–140. Disponível em: <<https://doi.org/10.1145/2504730.2504747>>.

MÖSER, M.; SOSKA, K.; HEILMAN, E.; LEE, K.; HEFFAN, H.; SRIVASTAVA, S.; HOGAN, K.; HENNESSEY, J.; MILLER, A.; NARAYANAN, A.; CHRISTIN, N. An empirical analysis of traceability in the Monero blockchain. **Proceedings on Privacy Enhancing Technologies**, v. 2018, n. 3, p. 143–163, 2018. Disponível em: <<https://doi.org/10.1515/popets-2018-0025>>.

PANINSKI, L. Estimation of entropy and mutual information. **Neural Computation**, v. 15, n. 6, p. 1191–1253, 2003. Disponível em: <<https://doi.org/10.1162/089976603321780272>>.

PERTSEV, A.; SEMENOV, R.; STORM, R. **Tornado Cash Privacy Solution: Version 1.4**. 2019. White paper. Disponível em: <<https://berkeley-efi.github.io/assets/material/Tornado%20Cash%20Whitepaper.pdf>>. Acesso em: 28 jul. 2026.

SERJANTOV, A.; DANEZIS, G. Towards an information theoretic metric for anonymity. In: **Privacy Enhancing Technologies (PET 2002)**. Berlin, Heidelberg: Springer, 2003. (Lecture Notes in Computer Science, v. 2482), p. 41–53. Disponível em: <https://doi.org/10.1007/3-540-36467-6_4>.

SERJANTOV, A.; DINGLEDINE, R.; SYVERSON, P. From a trickle to a flood: Active attacks on several mix types. In: **Information Hiding (IH**

2002). Berlin, Heidelberg: Springer, 2002. (Lecture Notes in Computer Science, v. 2578), p. 36–52. Disponível em: <https://doi.org/10.1007/3-540-36415-3_3>.

SMITH, G. On the foundations of quantitative information flow. In: **Foundations of Software Science and Computational Structures (FoSSaCS 2009)**. Berlin, Heidelberg: Springer, 2009. (Lecture Notes in Computer Science, v. 5504), p. 288–302. Disponível em: <https://doi.org/10.1007/978-3-642-00596-1_21>.

SOLANA FOUNDATION. **Address Lookup Tables**. 2026. Documentação oficial. Disponível em: <<https://solana.com/docs/advanced/lookup-tables>>. Acesso em: 28 jul. 2026.

SOLANA FOUNDATION. **Transaction Fees and the Compute Budget**. 2026. Documentação oficial. Disponível em: <<https://solana.com/docs/core/fees>>. Acesso em: 28 jul. 2026.

SOLANA FOUNDATION. **Transactions and Instructions**. 2026. Documentação oficial. Disponível em: <<https://solana.com/docs/core/transactions>>. Acesso em: 28 jul. 2026.

TÓTH, G.; HORNÁK, Z.; VAJDA, F. Measuring anonymity revisited. In: **Proceedings of the Ninth Nordic Workshop on Secure IT Systems (NordSec 2004)**. Espoo, Finland: [s.n.], 2004. p. 85–90. Disponível em: <<https://www.freehaven.net/anonbib/cache/THV04.pdf>>. Acesso em: 28 jul. 2026.

YAKOVENKO, A. **Solana: A New Architecture for a High Performance Blockchain**. 2018. White paper, v0.8.13. Disponível em: <<https://solana.com/solana-whitepaper.pdf>>. Acesso em: 28 jul. 2026.